Smart Wardrobe — Technical Overview

Educational reference • Bootcamp M4.3 • October 2026

Document purpose

This is an educational technical reference for the M4.3 RAG exercise. It distinguishes a proposed architecture from verified implementation details; no live service integration is implied.

1. Application context

Smart Wardrobe is a mobile-first wardrobe management concept with a demonstration workflow using Expo Go. The application aims to support clothing upload, organization, outfit planning, and future virtual try-on.

2. Structured clothing records

A suggested clothing item schema contains item_id, owner_id, image_reference, name, category, color, pattern, material, season, occasion, created_at, and updated_at. These are proposed fields rather than a confirmed current database schema.

3. AI classification workflow

An image-classification service may analyze a clothing photo and suggest category, color, pattern, material, and season. The app should store both the suggested values and user-approved corrections where practical.

4. Data storage option

Supabase is a candidate backend for authentication, relational clothing data, and image storage. This document does not assert that Supabase is currently connected or operational in the user's app.

5. API responsibilities

An application programming interface (API) can validate requests, associate items with authenticated users, request AI classification, and return structured results. Sensitive API keys must remain server-side and must never be embedded in a mobile client.

6. Modular virtual try-on

Virtual try-on should be isolated behind a replaceable provider interface. A future implementation could swap a commercial image-generation service for an open-source model without rewriting the entire wardrobe application.

7. RAG laboratory architecture

For the M4.3 exercise, two PDFs are split into text chunks. Each chunk includes metadata such as document title, section heading, source filename, and chunk number. OpenAI's text-embedding-3-small model can generate 1,536-dimensional vectors. Pinecone stores these vectors in a dedicated index and namespace.

8. Deterministic question answering

A deterministic RAG chain retrieves relevant document chunks before asking a language model to answer. Responses should cite the source document and section. If the retrieved material does not support an answer, the chain should explicitly state that the information is not available.

9. Separation of environments

The M4.3 laboratory lives in an independent folder and uses its own Pinecone namespace. It must not modify the main Smart Wardrobe application repository. A laboratory index is not the application's production database.

10. Limitations and security

RAG retrieval may miss relevant passages, and language models may still make mistakes. Evaluate answers against the source documents, limit access to private data, and keep API credentials out of source control.

11. Proposed implementation sequence

First, establish reliable clothing data entry and editable AI classifications. Second, improve outfit generation and user preferences. Third, prototype virtual try-on as a replaceable module. These are planning steps, not a claim that each milestone is complete.